МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Кафедра прикладної математики

ЗВІТ ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни «Бази даних»

на тему: «Реалізація каунтера з використанням PostgreSQL»

Виконав:
студент III курсу групи КМ-32,
спеціальність F1
Прикладна математика
Мурсюкаєв Дамір Андрійович

Київ – 2025

Результати тестування

Результати виконання коду, наведеного у Додатку А, для кожного сценарію наведені у табл. 1.

Таблиця 1 – результати тестування

Варіант	**Час виконання (с)**	**Фінальний counter**
Lost Update	10.41	10616
In-place Update	9.94	100000
Row-level Locking	14.66	100000
Optimistic Concurrency Control	62.01	100000

Результати виконання коду додаткового завдання, наведеного у Додатку Б, для кожного сценарію наведені у табл. 2. Для додаткового завдання був використаний метод In-place Update для оновлення counter.

Таблиця 2 – результати тестування додаткового завдання

Варіант	**Час виконання (с)**
Завжди перший рядок	9.87
Випадковий рядок	3.34

Висновки

Під час виконання даної лабораторної роботи було досліджено 4 варіанти оновлення значення каунтера для 10 потоків. Lost Update, як і було передбачено, загубив ~90% від всіх оновлень, оскільки потоки перезаписували один одного. In-place Update виявився найшвидшим і простим у реалізації, бо використовує атомарний запит на рівні SQL. Row-level Locking виявився повільнішим за In-place Update на 4 секунди через те, що потоки стоять у черзі. Optimistic Concurrency Control виявився

найдовшим у виконанні через те, що лише один з потоків виконує задачу першим, а інші повторюють спробу. Це створює велику кількість зайвих SELECT і UPDATE.

Для додаткового завдання, при оновленні одного фіксованого рядка всіма потоками час виконання склав 9.94 сек через високу конкуренцію за блокування. При оновленні випадкових рядків – 3.98 сек, оскільки потоки працюють паралельно без конфліктів. Таким чином, при багатопотоковому доступі конкуренція за рядок має більший вплив на продуктивність ніж пошук за індексом.

Додаток А. Код програми

```
import psycopg2
import threading
import time

DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "lab2_db",
    "user": "lab2_user",
    "password": "lab2_pass"
}

THREADS = 10
ITERATIONS = 10_000


def get_connection():
    return psycopg2.connect(**DB_CONFIG)


def reset_counter():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE user_counter SET counter = 0, version = 0 WHERE user_id =
1")
    conn.commit()
    cur.close()
    conn.close()


def get_counter():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT counter FROM user_counter WHERE user_id = 1")
    value = cur.fetchone()[0]
    cur.close()
    conn.close()
    return value


def lost_update_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        cur.execute("SELECT counter FROM user_counter WHERE user_id = 1")
        counter = cur.fetchone()[0]
        counter += 1
        cur.execute("UPDATE user_counter SET counter = %s WHERE user_id = 1",
(counter,))
        conn.commit()
    cur.close()
    conn.close()


def inplace_update_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        cur.execute("UPDATE user_counter SET counter = counter + 1 WHERE user_id =
1")
        conn.commit()
    cur.close()
    conn.close()
```

```
def row_lock_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        cur.execute("SELECT counter FROM user_counter WHERE user_id = 1 FOR UPDATE")
        counter = cur.fetchone()[0]
        counter += 1
        cur.execute("UPDATE user_counter SET counter = %s WHERE user_id = 1",
(counter,))
        conn.commit()
    cur.close()
    conn.close()


def optimistic_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        while True:
            cur.execute("SELECT counter, version FROM user_counter WHERE user_id =
1")
            counter, version = cur.fetchone()
            counter += 1
            cur.execute(
                "UPDATE user_counter SET counter = %s, version = %s WHERE user_id =
%s AND version = %s",
                (counter, version + 1, 1, version)
            )
            conn.commit()
            if cur.rowcount > 0:
                break
    cur.close()
    conn.close()


def run_scenario(name, worker_fn):
    reset_counter()
    threads = [threading.Thread(target=worker_fn) for _ in range(THREADS)]
    start = time.time()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.time() - start
    final = get_counter()
    print("--------------------------------+---------------+--------------------")
    print(f"{name:<30}  | {elapsed:<13.2f} | {final}")


def main():
    print("Варіант                         | Час виконання |  Фінальний counter")
    run_scenario("Lost Update", lost_update_worker)
    run_scenario("In-place Update", inplace_update_worker)
    run_scenario("Row-level Locking", row_lock_worker)
    run_scenario("Optimistic Concurrency Control", optimistic_worker)


if __name__ == "__main__":
    main()
```

Додаток Б. Код додаткової програми

```
import psycopg2
import threading
import time
import random

DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "lab2_db",
    "user": "lab2_user",
    "password": "lab2_pass"
}

THREADS = 10
ITERATIONS = 10_000
MAX_USER_ID = 100_000


def get_connection():
    return psycopg2.connect(**DB_CONFIG)


def reset_counters():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE user_counter_big SET counter = 0, version = 0")
    conn.commit()
    cur.close()
    conn.close()


def first_row_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        cur.execute("UPDATE user_counter_big SET counter = counter + 1 WHERE user_id
= 1")
        conn.commit()
    cur.close()
    conn.close()


def random_row_worker():
    conn = get_connection()
    cur = conn.cursor()
    for _ in range(ITERATIONS):
        user_id = random.randint(1, MAX_USER_ID)
        cur.execute("UPDATE user_counter_big SET counter = counter + 1 WHERE user_id
= %s", (user_id,))
        conn.commit()
    cur.close()
    conn.close()


def run_scenario(name, worker_fn):
    reset_counters()
    threads = [threading.Thread(target=worker_fn) for _ in range(THREADS)]
    start = time.time()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.time() - start
    print("--------------------+--------------------")
```

```
    print(f"{name:<20}| {elapsed:.2f}")


def main():
    print("Варіант              | Час виконання (с)")
    run_scenario("Завжди перший рядок", first_row_worker)
    run_scenario("Випадковий рядок", random_row_worker)


if __name__ == "__main__":
    main()
```